CS23532 – COMPUTER NETWORKS
EX 13: Own Ping Program

AIM :

To implement our own ping program using raw sockets and the Internet Control Message Protocol (ICMP).

ALGORITHM :

1. Import the socket, struct, time, sys and os modules.

2. Resolve the destination host name to an IP address.

3. Create a raw socket using the ICMP protocol and set a timeout.

4. Build an ICMP Echo Request packet: type = 8, code = 0, the identifier (process id), the sequence number and a 32-byte data payload.

5. Calculate the Internet checksum of the header and payload and insert it into the packet.

6. Send the packet to the destination and note the sending time.

7. Wait for the reply. On receiving an ICMP Echo Reply (type = 0) with the same identifier and sequence number, calculate the round trip time and the TTL and display them. If no reply arrives before the timeout, display “Request timed out.”

8. Repeat the steps for four packets, one per second.

9. Display the ping statistics: packets sent, received and lost, and the minimum, maximum and average round trip times.

PROGRAM :

my_ping.py

```
import socket
import struct
import time
import sys
import os

def checksum(data):
    if len(data) % 2:
        data += b'\x00'
    total = 0
    for i in range(0, len(data), 2):
        total += (data[i] << 8) + data[i + 1]
    total = (total >> 16) + (total & 0xFFFF)
    total += total >> 16
    return ~total & 0xFFFF
```

```
def build_packet(seq):
    pid = os.getpid() & 0xFFFF
    header = struct.pack('!BBHHH', 8, 0, 0, pid, seq)
    payload = b'abcdefghijklmnopqrstuvwabcdefghi'
    chk = checksum(header + payload)
    header = struct.pack('!BBHHH', 8, 0, chk, pid, seq)
    return header + payload

def ping(host, count=4, timeout=2):
    dest = socket.gethostbyname(host)
    sock = socket.socket(socket.AF_INET, socket.SOCK_RAW,
socket.IPPROTO_ICMP)
    sock.settimeout(timeout)
    pid = os.getpid() & 0xFFFF
    print("Pinging %s [%s] with 32 bytes of data:" % (host, dest))
    times = []
    for seq in range(1, count + 1):
        sock.sendto(build_packet(seq), (dest, 0))
        start = time.time()
        try:
            while True:
                data, addr = sock.recvfrom(1024)
                icmp_type, code, chk, rid, rseq = struct.unpack('!BBHHH',
data[20:28])
                if icmp_type == 0 and rid == pid and rseq == seq:
                    rtt = (time.time() - start) * 1000
                    ttl = data[8]
                    times.append(rtt)
                    print("Reply from %s: bytes=%d time=%.2fms TTL=%d" %
(addr[0], len(data) - 28, rtt, ttl))
                    break
        except socket.timeout:
            print("Request timed out.")
        time.sleep(1)
    sock.close()
    lost = count - len(times)
    print("\nPing statistics for %s:" % dest)
    print("    Packets: Sent = %d, Received = %d, Lost = %d (%d%% loss)" %
(count, len(times), lost, lost * 100 // count))
    if times:
        print("Round trip times in milli-seconds:")
        print("    Minimum = %.2fms, Maximum = %.2fms, Average = %.2fms" %
(min(times), max(times), sum(times) / len(times)))

if __name__ == '__main__':
    target = sys.argv[1] if len(sys.argv) > 1 else 'localhost'
    ping(target)
```

INPUT AND OUTPUT :

Input: The program uses a raw socket, so it must be run with administrator (root) privileges. The destination host is given as the command line argument: localhost

Output

```
$ sudo python3 my_ping.py localhost
Pinging localhost [127.0.0.1] with 32 bytes of data:
Reply from 127.0.0.1: bytes=32 time=0.01ms TTL=64
Reply from 127.0.0.1: bytes=32 time=0.03ms TTL=64
Reply from 127.0.0.1: bytes=32 time=0.04ms TTL=64
Reply from 127.0.0.1: bytes=32 time=0.03ms TTL=64

Ping statistics for 127.0.0.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss)
Round trip times in milli-seconds:
    Minimum = 0.01ms, Maximum = 0.04ms, Average = 0.03ms
```

RESULT:

Thus, our own ping program was implemented using raw sockets and ICMP, and it was executed successfully.